(Blind) Multimodal Bot Detection on X

Hamza Rashid
McGill University
COMP 400

***Abstract*—This research investigates multimodal bot detection methods on X, in a setting where the labels are not known a priori, and graph data is not available. We present a comprehensive evaluation of bot detection by benchmarking classical machine-learning baselines and feature engineering techniques, and later develop deep multimodal architectures. Our work begins with conventional classifiers—a random forest on text, temporal, and topic-distribution features, yielding moderate baseline performance. We next investigate how humans and bots differ in their temporal posting behaviour, finding little distinction except for outliers. As feature engineering approaches are easily deceived by sophisticated bots, we move on to multimodal architectures with an emphasis on DNA-inspired behaviour modeling. For this, we embed the user description with TwHIN-BERT and model posting behavior with two DNA formulations, one that captures content across posts, and another capturing inter-post times. We find that CNN embeddings of inter-post time DNA may capture deep patterns in time series data that conventional methods miss. We investigate three methods for processing the modalities and final fusion; a GMU (gated multimodal unit), a cross-attention mechanism, and a sparsely gated MOE (mixture-of-experts) with a self-attention fusion layer and a convolution learned on attention weights that addresses manipulated features. We find that the MOE performs best, and gather counter-intuitive results along the way. Namely, the MOE is the smallest of our multimodal models, and does not require any treatment of the class imbalance found in bot detection datasets. More broadly, we find that a multimodal approach to bot detection improves robustness against sophisticated bots, and that a MOE architecture helps in addressing the diversity of user communities on X.**

I. Introduction

Social media platforms like X (formerly Twitter) have enabled unprecedented global connectivity. As an open platform is vulnerabe, X is fertile ground for malicious content, often originating from automated accounts—bots—that manipulate discourse, spread misinformation, or artificially amplify content. Detecting and mitigating such bot activities remains a challenging yet essential task for preserving authentic online interactions.

Traditional approaches to bot detection typically rely on supervised classification methods trained on manually labeled datasets. Yet, in real-world scenarios, labeled data are often scarce, outdated, or non-representative of the evolving behaviors exhibited by sophisticated bots. Consequently, there is a pressing need for detection methods that generalize beyond known bot patterns, especially in contexts where labeled data is not readily available.

In this report, we address this challenge by evaluating a range of bot detection strategies, beginning with classical machine-learning techniques enhanced by extensive feature engineering. We initially employ a random forest classifier incorporating textual features, temporal posting patterns, and inferred topic distributions, achieving moderate baseline performance. Our analysis reveals that traditional temporal features often fail to effectively distinguish sophisticated bots from genuine users, underscoring the limitations of classical approaches.

Building on these insights, we introduce deep multimodal architectures designed to exploit richer behavioral patterns. Inspired by the concept of digital DNA, our method integrates user descriptions embedded via TwHIN-BERT and models posting behavior through two distinct DNA formulations: one capturing the content across posts and another encoding inter-post timing patterns. In addition to the known efficacy of content DNA (Ilias et al., 2023), our experiments demonstrate that CNNs applied to temporal-DNA-generated images effectively uncover nuanced behavioral patterns overlooked by conventional methods. As most of the frontier literature overlooks temporal bevahiour, this result signifies that it could be worth further investigation.

We further explore several fusion techniques to integrate multimodal signals, including gated multimodal units (GMUs), cross-attention mechanisms, and a novel application of a sparsely gated mixture-of-experts (MOE) (Shazeer et al., 2017) architecture. Our results highlight the MOE model's superior performance and efficiency, notably without necessitating explicit strategies to address class imbalance. Overall, our research emphasizes the robustness of multimodal approaches and illustrates the potential of MOE architectures to adapt effectively to the diverse user communities found on X.

II. Dataset and Experimental Setup

Our experiments were conducted within the "Bot Or Not" (B.O.N.) competitive infrastructure, designed specifically for assessing bot detection strategies in realistic social media scenarios. As it is currently in the development phase, the conditions outlined in this report are subject to change in future versions. This environment featured interactions between bot and detector teams, simulating genuine activities on the X platform. The primary goal was to evaluate the effectiveness of detection methodologies under conditions closely resembling live social media dynamics.

A. Data Collection and Characteristics

The dataset comprised real posts gathered from the X API across multiple collection sessions, each lasting the same duration but occurring on different days. For each session,

posts were initially collected based on predefined sets of topical keywords relevant to that session. After gathering initial posts, users flagged as obvious bots were removed, and the dataset was expanded by including all additional posts from the remaining users within the session period.

The preliminary data collection process can summarized as follows:

- Text-only Posts: Posts containing images, videos, or memes were excluded.
- Static Metadata: Dynamic metadata such as likes, comments, and follower counts were excluded to maintain consistency and simplicity.
- Anonymized Links and Mentions: Hyperlinks were standardized to "https://t.co/twitter_link", and user mentions were anonymized to "@mention" to prevent easy verification of authenticity.
- Activity Thresholds: Each included user produced between 10 and 100 posts during the session period, ensuring active participation.

Datasets were structured into two main JSON formats:

- Posts Dataset: Included text content, timestamp, unique post ID, author ID, and language.
- Users Dataset: Contained user-specific metadata, including user ID, username, display name, profile description, location, tweet count, and a calculated activity-based z-score.

B. Session Structure

Sessions were divided into multiple sequential sub-sessions to mimic the real-time flow of social media feeds, gradually introducing new posts to participants. Bot teams generated content incrementally across these sub-sessions, with the requirement to post between 10 and 100 posts per bot by session completion. Detector teams subsequently analyzed the fully compiled datasets post-session to identify bots, providing both binary verdicts and confidence scores for each account.

C. Technical Setup

Participant submissions (bot and detector code) were managed via public GitHub repositories cloned into Docker images deployed on EC2 instances, ensuring standardized computing resources and fair evaluation conditions. Submissions were required to execute fully within a one-hour timeframe, after which automated shutdown procedures terminated any ongoing processes.

III. Methodology

Our approach involved a systematic progression from classical machine learning methods to multimodal neural-based architectures, aimed at overcoming limitations observed at each stage. Our primary goal was to model user behaviour from a variety of perspectives since nontrival bots manipulate features in at least one modality (e.g., tweet content, posting times, or interactions such as retweets, likes, and follower and following relationships).

A. Baseline - Random Forest

Our baseline method involved training a Random Forest classifier using various handcrafted and learned features from user posts. We organized our experiments across multiple data collection sessions to assess how well feature-based models generalize across batches of social media activity. The sub-approaches below correspond to progressively enriched feature sets tested on different sessions. The primary motivation for employing a random forest is that it enables rapid feature exporation due to its flexibility (allows multiple data types in its feature set), and its proven efficacy in bot detection, most notably in Botometer (Davis et al., 2016).

1) Session 11 – Embeddings + LDA: We began with a straightforward approach combining sentence-level embeddings with LDA (Latent Dirichlet Allocation)-based topic distributions. Each user's posts were aggregated into a single document, from which features were extracted.

Implementation Summary:

- Text Cleaning: URLs, mentions, and hashtags were anonymized. Tokenization, lemmatization, and stopword filtering were performed using spaCy.
- Embedding: Posts were embedded using all-MiniLM-L6-v2 from SentenceTransformers.
- Topic Modeling: CountVectorizer was used for bag-of-words representations, followed by LDA with 10 topics.
- Sentence embeddings (384-dim) were concatenated with LDA topic distributions.
- Classifier: A balanced Random Forest with 100 trees was trained and evaluated using stratified 80/20 train/test splits.

Result: The model exbihibted low confidence and marked all samples as negatives. This likely owes to its reliance on predefined topics, which vary across sessions, resulting in sparse vectors on unseen data.

Limitations:

- Did not use individual tweets as independent samples, thereby lacking granulrity.
- Static topic modeling limited generalization to new sessions.

2) Session 12 – Mean Embedding + BERTopic + Temporal Stats: In this session, we shifted to using per-tweet embeddings and adopted BERTopic for more dynamic topic modeling. Additional temporal statistics were introduced to capture behavioral rhythms.

Implementation Summary:

- Tweet-Level Processing: Individual tweets were embedded and clustered into topics using BERTopic with HDBSCAN.
- Aggregation: User-level features were computed via:
  - Mean of sentence embeddings
  - Normalized topic distribution (padded to 361 dimensions)
- Temporal features: mean and standard deviation of post timestamps

- Static features: Post count
- Classifier: The same Random Forest model from Session 11 was applied.

Result: The use of dynamic topic modeling and behavioral signals significantly improved classification performance. Bot recall improved due to BERTopic's unsupervised generalization across sessions.

Limitations:

- Topic dimension padding was static and could introduce noise.
- Temporal features were still limited to coarse statistics.

3) Session 13 – Enhanced Temporal Behavior Modeling: Session 13 introduced a novel temporal feature: duplicate post timestamps within bucketed time intervals, designed to capture artificial timing artifacts used by bots.

Implementation Summary:

- Building on Session 12, we added:
- Inter-post Interval Statistics: Mean and standard deviation of time gaps between posts.
- Bucketed Duplicate Counts: Time was split into 10 buckets, and duplicate post counts were computed per bucket.
- Features Used: Combined embeddings, topic distributions, post frequency, inter-post intervals, and bucketed duplicates.

Result: This was the strongest-performing Random Forest baseline. The bucketed duplicates provided a clear signal in identifying bots with repeated behavior patterns or scheduled posts.

Limitations:

- Still relied heavily on session-specific BERTopic clusters.
- Lacked modeling of inter-feature dependencies (e.g., topic-time correlations).

4) Session 14 – Topic Statistics + Temporal Entropy + Intra-Topic Similarity: This final Random Forest setup expanded feature engineering to include statistical, temporal, and semantic regularities observed in bot behavior. The design aimed to encode not just what a user posts, but also how, and when, consistently.

Implementation Summary:

- Refined Preprocessing: Hashtags were partially preserved (`#arena` → `<HashtagMention:arena>`), preserving topic cues often used by bots.
- Normalized Embeddings: Sentence embeddings (MiniLM-L6-v2) were explicitly normalized to prevent variance from dominating downstream signals.
- Topic Modeling: BERTopic + HDBSCAN, followed by:
  - Topic Histogram (topic mention counts across topics)
  - Summary Statistics: Mean, variance, std, entropy, and dominance ratio across topic indices.
- Temporal Signals:
  - Mean, std, and variance of time between posts.
  - Bucketed duplicate timestamps (tolerance-aware) to flag spam bursts
- Intra-topic Semantic Consistency:
  - Mean, median, and std of pairwise cosine similarities across tweets within each user-topic group.

Result: In session 14, the bots became particularly hard to distinguish upon manual inspection. Although this model outperformed our previous attempt under the same training conditions when applied to the previous session, its performance was worse on the latest.

Limitations:

- Feature dimensionality increased significantly, potentially stressing interpretability.
- BERTopic remains sensitive to session-specific clustering variance.

B. Summary and Reflection on Random Forest Baseline

Our progression through four sessions using Random Forest classifiers illustrates the evolving challenge of distinguishing bots from humans on X through feature engineering alone.

What Went Right:

- Rich Feature Design: Session 14 demonstrates that carefully designed user-level features (temporal, semantic, topical) can yield strong performance using classical models despite evolving bots.
- Dynamic Topic Modeling: Transitioning from static LDA to BERTopic+HDBSCAN enabled models to better capture cross-session topical drift.
- Behavioral Signals Help: Features such as inter-post intervals and intra-topic semantic consistency offered important signals for detecting scripted or bursty behavior patterns.

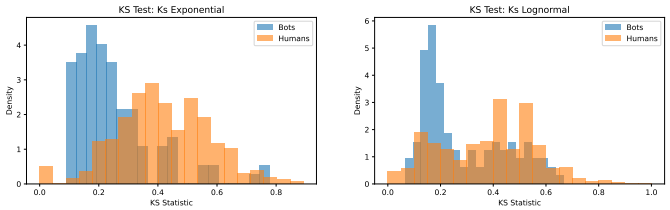
What Went Wrong:

- Sensitivity to Session Artifacts: A non-pretrained BERTopic model was applied to each session, which likely created inconsistencies in how topic-features were interpreted by the classifier.
- Appropriateness of features: Despite being a universal approximator, Random Forest's are typically geared towards categorical features; introducing several related continuous features like embeddings can introduce noise.
- Limited Generalization: Although high accuracy may have been achieved on per-session splits, the performance likely degraded when faced with unseen session distributions or bot styles.
- Feature Brittleness: Despite capturing many dimensions, these features are often brittle against slight changes in bot strategies (e.g., using synonyms or reworded hashtags).
- Scalability Concerns: Computing transformer embeddings, pairwise similarities, multiple topic stats, and padded histograms scales poorly for large user datasets.

While the Random Forest baseline allowed us to test many ideas rapidly, its limitations became evident in detecting sophisticated or adaptive bots (increasing complexity of feature engineering was needed to keep up with the bots), motivating the shift to multimodal neural architectures whose underlying features can adapt to evolving behaviours on X.

C. Temporal Feature Analysis via KS Tests

To assess the discriminative power of static temporal features, we computed inter-post interval distributions for each user and performed Kolmogorov–Smirnov (KS) tests against four theoretical models: exponential, normal, lognormal, and Weibull. Figures 1a-1d display the resulting KS statistic distributions for bots versus humans. A lower KS value indicates a better fit to the assumed distribution, while a higher value implies greater deviation from the theoretical model.



(a) Exponential model (b) Lognormal model

(c) Normal model (d) Weibull model

Fig. 1: KS statistic distributions for inter-post intervals under various distributional fits.

While a formal ablation study would be needed to confirm the hypothesis, the KS-based analysis supports our intuition that static temporal features are not reliable indicators of bot-like activity. While the majority of bots fit the theoretical distributions more closely, the substantial overlap in KS statistics across all four models suggests that these static summaries cannot capture the complex timing behaviors exhibited by sophisticated bots.

This finding, in part, motivates our shift toward deep multimodal architectures, where temporal information is represented in a more dynamic manner and learned jointly with other behavioral signals, enabling the model to uncover subtle, context-dependent patterns that handcrafted temporal features miss.

D. Multimodal Architectures

Recognizing the constraints of feature engineering, we explored multimodal neural architectures directly modeling sequences and cross-modal interactions inspired by recent literature *"Multimodal Detection of Bots on X (Twitter) using Transformers"* (Ilias et al., 2023), and *"BotMoE: Twitter Bot Detection with Community-Aware Mixtures of Modal-Specific Experts"* (Liu et al., 2023). Our modalities include transformer-based description and tweet embeddings alongside digital DNA representations (content and temporal).

Evaluated architectures include:

- **Gated Multimodal Unit (GMU)**: Dynamically weighting modalities.
- **Cross-Attention Fusion**: Learns cross-modal interactions through multihead attention.
- **Mixture-of-Experts (MOE)**: Subsets of the models specialize in different user types, with attention-based fusion and a learned convolution over attention weights.

These architectures not only provide greater modeling capacity, but also offer more robustness against feature manipulation, as they do not rely on manually crafted statistics. Instead, they operate directly over sequences of observed user behavior and learn to extract structure.

1) DNA Embedding Extraction: To model user behavior beyond text embeddings, we construct symbolic sequences known as digital DNA, across the user's timeline chronologically, representing posting content, timing, and emoji usage. These sequences are then converted into images and embedded via a CNN encoder. In all cases, the resulting character sequence is transformed into an image as follows:

1) Each symbol is mapped to an 8-bit grayscale intensity value (e.g., `N` $\mapsto$ 0, `U` $\mapsto$ 64, ..., `X` $\mapsto$ 255).
2) The sequence of grayscale values is reshaped into a square matrix: if the sequence has length L, the smallest $n \in \mathbb{N}$ such that $n^2 \geq L$ is selected. The sequence is zero-padded to length n^2 and reshaped to $(n \times n)$.
3) This grayscale image is resized depending on the CNN encoder, using nearest-neighbor interpolation.
4) The image is then converted to RGB format by replicating the grayscale channel, and the resulting tensor has shape: $3 \times$ `image_width` $\times$ `image_height`.
5) The tensor is normalized using ImageNet statistics and passed through a pretrained CNN encoder.
6) The output features are either (1) average pooled and projected into a fixed embedding space, or (2) directly extracted from the last hidden layer of the CNN encoder.

Content DNA. Each tweet is mapped to a symbol based on the type of entities it contains:

- `N` — No entities (plain text)
- `U` — Contains a URL
- `H` — Contains one or more hashtags
- `M` — Contains one or more mentions
- `X` — Contains a combination of multiple entity types

Time DNA. Time DNA encodes inter-post intervals by assigning symbols based on the time elapsed between consecutive tweets:

- `n` — First post (no delta)
- `o` — $<$ 1 second
- `s` — $\geq$ 1 second and $<$ 1 minute
- `m` — $\geq$ 1 minute and $<$ 1 hour
- `h` — $\geq$ 1 hour and $<$ 6 hours
- `u` — $\geq$ 6 hours and $<$ 12 hours
- `x` — $\geq$ 12 hours

Emoji DNA. We categorize emojis in tweets using a precompiled lookup table. Each tweet is assigned a category based on its emojis:

- `S` — Smileys & Emotion
- `P` — People & Body
- `A` — Animals & Nature
- `F` — Food & Drink
- `T` — Travel & Places
- `R` — Activities
- `O` — Objects
- `Y` — Symbols
- `L` — Flags
- `N` — No emoji present
- `X` — Multiple emoji categories

In all cases, these DNA images serve as structured behavioral fingerprints, allowing CNNs to learn latent patterns over the visualized behavior sequences. Depending on the architecture, the extracted embeddings are used individually or fused with other modalities.

2) Gated Multimodal Unit (GMU): The GMU allows the network to dynamically control how much each modality contributes to the final hidden representation. The output is used (with possibly further transformations) downstream for classification.

In the bimodal setting, where we consider two input modalities $\mathbf{x}_v$ and $\mathbf{x}_t$, the equations governing the GMU are as follows:

$$\mathbf{h}_v = \tanh(\mathbf{W}_v \cdot \mathbf{x}_v) \tag{1}$$

$$\mathbf{h}_t = \tanh(\mathbf{W}_t \cdot \mathbf{x}_t) \tag{2}$$

$$\mathbf{z} = \sigma(\mathbf{W}_z \cdot [\mathbf{x}_v, \mathbf{x}_t]) \tag{3}$$

$$\mathbf{h} = \mathbf{z} * \mathbf{h}_v + (1 - \mathbf{z}) * \mathbf{h}_t \tag{4}$$

$$\mathbf{\Theta} = \{\mathbf{W}_v, \mathbf{W}_t, \mathbf{W}_z\} \tag{5}$$

Here, $\mathbf{h}_v$ and $\mathbf{h}_t$ are hidden states learned on modalities $\mathbf{x}_v$ and $\mathbf{x}_t$, $[.,.]$ denotes concatenation, and $\mathbf{\Theta}$ denotes learnable parameters. Cross-modal interactions are learned in the parameters $\mathbf{W}_z$, the weighting mechanism $\mathbf{z}$ then applies a sigmoid activation σ on the projected modalities to determine how much each one contributes to the final fused representation. In practice, implementations tend to depart[1] from some of the original equations, including ours.

Resembling the work of *Ilias et al.*, our intial efforts with the GMU used the following modalities:

- User description embedding from TwHIN-BERT.
- Content-DNA embedding from MobileNetV2.

The first iteration consisted of a two layer classification head applied to the GMU's output. This model marked all samples as negatives, and several adjustments were needed. Working backwards from our final iteration, we reason that the first attempt was ineffective because it did not use normalization to deal with input variation and was unable to learn cross-modal interactions effectly due to its simple linear gating mechanism.

a) Architectural Progression Toward the Final GMU.: To improve the initial GMU, we developed two refined variants:

- **GMUFusionWithNorm:** Introduced modality-specific projections and *LayerNorm* to balance their scales. Normalization was applied after fusion to stabilize the representation.
- **GMUFusionWithGlobalSkip:** Added a global residual connection that projected raw modality embeddings directly to the hidden space, and applied modality-specific dropout.

b) Incorporating Time-DNA and Its Impact.: While refining our GMUs, time-DNA embeddings were introduced as a third modality. This addition improved performance, supporting our intuition that a dynamic temporal feature can help capture richer distinctions between bots and humans.

While our GMU iterations demonstrated nontrivial improvements, development required numerous adjustments to stabilize training and optimize performance. This raised concerns about generalizability; would such a carefully balanced architecture transfer effectively to different modalities and datasets?

Given these concerns, we transitioned to cross-modal attention mechanisms, hypothesizing that they could acheive nontrivial performance with fewer architectural tweaks.

3) Cross-Attention: Equipped with three modalities (user description, content-DNA, and time-DNA), it would have been computationally expensive to generalize the work of *Ilias et al.*, which would require six multihead-attention layers to consider each cross-modal interaction in both directions. A natural alternative is to employ a single self-attention block where each token corresponds to one of the modalities. We found little success with this approach, but given that it plays a crucial role in our final MOE model, it is certainly worth further exploration.

As a workaround, we fuse modalities in two stages:

- Apply a simple GMU to the DNA modalities to create an intermediate representation of fused-DNA.
- Apply cross-attention layers to the description and fused-DNA in both directions.

To perform cross-modal attention, we employ two multihead-attention layers, essentially computing scaled dot-product attention (Vaswani et al., 2017)

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{6}$$

in one direction where the description modality is the query (Q) and fused-DNA provides the key (K) and value (V), and another direction in which the modalities swap roles. Here, d_k denotes the dimensionality of the query and key. A mean pooling is applied to both attention layers' outputs to form a final fused representation.

With the fused representation, we began with a two layer classification head and iteratively improved our architecture. This required less refinements than our GMU approach.

[1]Arevalo *et al.*'s own demo https://github.com/johnarevalo/gmu-mmimdb/blob/master/model.py; omits separate projections for each modality and applies the nonlinearity directly to the inputs.

a) Session 16 - Description, Content, Time: For session 16, we employed the cross-attention approach on user description embeddings and CNN-encoded content and time DNA sequences.

Architecture: Along with the previously outlined steps, The design follows a structured flow employing conventional methods such as residual connections and layer normalization:

- **Modality Preparation:**
  - The user description embedding is projected into a hidden space.
  - Content and time DNA sequences are fused via the GMU as stated before.
- **Cross-Modal Fusion:**
  - **Pre-Layer normalization** (Xiong et al., 2020) and **residual connections** are applied at each attention stage to stabilize training and preserve modality-specific signals.
- **Feature Refinement:**
  - The two cross-attention outputs are concatenated into a tiny sequence and fed through a single position-wise FFN, enabling direct interaction between modalities while sharing parameters for better regularization.
  - A residual connection around the FFN plus a preceding LayerNorm ensure stable feature propagation and consistent scaling.
- **Aggregation and Prediction:**
  - **Global average pooling** condenses the multi-token output into a single fused vector.
  - A **two-layer classification head** generates the final logits for bot detection.

Strengths:

- *Hierarchical fusion*: In addition to the bidirectional cross-attention steps, GMU's gated combination of content and time embeddings produces richer joint representations than simple concatenation alone.
- *High expressiveness*: Beyond the cross-attention fusion, treating each modality as its own token and then merging them enables the model to capture subtle behavioral cues (e.g., burst patterns or nuanced profile signals) that flatter architectures miss.

Weaknesses:

- *Overparameterization risk*: Multiple attention layers plus gating mechanisms add significant complexity, making the model prone to overfitting without sufficient data or strong regularization.
- *Hyperparameter sensitivity*: Stable training depends on careful tuning of learning rate, dropout rate, number of heads, gating biases, etc.; suboptimal settings can lead to slow convergence or degraded performance.

Performance: The model underperformed relative to some of our random forest attempts. However, the bots had improved significantly by session 16, and a drop in performance was expected given the model's complexity and the limited fine-tuning conducted at this stage.

Conclusion: While the session 16 model introduced a promising fusion paradigm combining gated mechanisms and cross-modal attention, it fell short in performance due to its tuning demands. Nonetheless, this model performed well given its complexity and lack of tuning, and it set the stage for our future attempts.

IV. Discussion

blah blah....

V. Conclusion and Future Work

blah blah...

References

Clayton A. Davis, Onur Varol, Emilio Ferrara, Alessandro Flammini, and Filippo Menczer. Botornot: A system to evaluate social bots. In *Proceedings of the 25th International Conference Companion on World Wide Web*, pages 273–274. ACM, 2016.

Loukas Ilias, Ioannis Michail Kazelidis, and Dimitris Th. Askounis. Multimodal detection of bots on x (twitter) using transformers. *arXiv preprint arXiv:2308.14484*, 2023. URL https://arxiv.org/abs/2308.14484.

Yuhan Liu, Zhaoxuan Tan, Heng Wang, Shangbin Feng, Qinghua Zheng, and Minnan Luo. Botmoe: Twitter bot detection with community-aware mixtures of modal-specific experts. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '23, pages 485–495, Taipei, Taiwan, July 2023. ACM. doi: 10.1145/3539618.3591646.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *CoRR*, abs/1701.06538, 2017. URL http://arxiv.org/abs/1701.06538.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 5998–6008. Curran Associates, Inc., 2017. URL https://papers.nips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. *arXiv preprint arXiv:2002.04745*, 2020. URL https://arxiv.org/abs/2002.04745.